

Array

Jundale Sir

Page No.

Date

* Array -

It is a collection of elements / group of elements having similar data type & referred by a specific name.

Note - In python there is no built in data type Array. So, in python array looks same as lists.

eg Simple example of an array -

eg `a = [1, 2, 3, 4, 5]` # simple array

Syntax - `array_name = [ele1, ele2, ---]`

eg `a = [1, 2, 3, 4, 5]`

* Accessing element of an array -

eg `print(a[2])` # $0 \rightarrow 3$ it prints element 3.

* Assigning elements to array by scanning from user -

`a = []` # declⁿ of an array a (its blank)

for i in range(0, 5):

`ele = int(input())`

`a.append(ele)` # where append() # method is used to add / append # elements at end

`print(a)` # printing array

* Declaring an array using array module -

Syntax - `arrayname = array (typecode, [initializers])`

where, array - it is a name of module which is provided by an ~~array~~ python.

typecode - it is code used to define type of value the array hold.

- There are following types of typecode

typecode → "b" - int - mini. size 1 byte ← python it is said ← size

"B" - int - 1

"u" - unicode character - 2

"h" - int - 2

"H" - int - 2

"i" - int - 2 (~~signed~~ (signed int))

"I" - int - 2 (unsigned int)

"l" - int - 4 (signed long)

"L" - int - 4 (unsigned long)

"q" - int - 8 (signed long long)

"Q" - int - 8 (unsigned long long)

"f" - float - 4 (float)

"d" - float - 8 (double)

initializers - a set of similar type of elements.

from array import * # adding module

eg. `a = array ("i", [1, 2, 3, 4, 5])`

`print (a)`

or `for i in a:`

`print (i)`

* modifying elements of an array at particular index position -

eg In above example to modify array at index position '0'

eg $a[0] = 1000$ # modify element at 0th index

* Adding element into array -

① insert() - it is used for insert element at particular index position in array

Syntax

array name . insert (index, element)

eg $a = [1, 2, 3, 4, 5]$

$a.insert(3, 20000)$ # insert element at index 3

print(a)

o/p - 1, 2, 3, 20000, 4, 5

② append() - it is used to add/append element into array at last position

Syntax

array-name . append (element)

eg $a = [1, 2, 3, 4, 5]$

$a.append(10000)$ # appends / add
10000 element at end

print(a)

o/p - [1, 2, 3, 4, 5, 10000]

* Removing elements from array -

① remove() - it is used for remove only one element from array (particular element from array)

Syntax - array_name.remove(element)

eg a = [10, 20, 30, 40, 50]
a.remove(30) # removes element 30
print(a)

② pop() - it is used to remove element from array at particular index position / remove last element by default

eg a = [10, 20, 30, 40, 50]
eg a.pop() # removes default last element
eg a.pop(2) # removes element at index position 2

Syntax : array.pop()

or array_name.pop(index)

eg a.pop()
or a.pop(2)

* Reversing elements of array -

reverse() - it is used to reverse elements of an array

Syntax -

array_name.reverse()

eg a = [10, 20, 30, 40, 50]
a.reverse() # reverse array
print(a)

* sort() method - it is used for sorting an array elements by ascending order

eg `a = [10, 5, 4, 100, 20]`

`a.sort()` # sorting array

`print(a)` # o/p - 4, 5, 10, 20, 100

* clear() method - it is used for to clear elements of an array

Syntax `array-name.clear()`

eg `a = [10, 20, 30, 40, 50]`

`a.clear()` # clear all elements of

`print(a)` an array

* Copying elements of one array to another array -

① by using assignment (=) operator -

Syntax `old-array-name = new_array_name`

eg `a = [1, 2, 3, 4, 5]`

`b = a` # copy a to b

`print(b)`

② by using copy() method -

Syntax - `new_array = old_array.copy()`

eg `a = [1, 2, 3, 4, 5]`

`b = a.copy()`

`print(b)`

* count() method - it is used to count the specified element will come how many times in array.

Syntax - arrayname.count(element_name)

eg $a = [1, 2, 1, 3, 1, 5]$

$b = a.count(1)$ # count 1 comes
 $print(b)$ how many times

* extend() method - it is used to add sub array at the end of existing array

Syntax - array.name.extend(new subarray)

eg $a = [10, 20, 30]$

$print(a)$

$b = [50, 60, 70]$ # new array

$a.extend(b)$ # extend a by adding
b

$print(b)$ # o/p $[10, 20, 30, 50, 60, 70]$

* index() method - it is used to print index of particular element in an array.

Syntax variable = array_name.index(element)

eg $a = [10, 20, 30, 40]$

$c = a.index(30)$ # access index of 30

$print(c)$ # o/p - 2

* Types of an array -

① One dimensional array -

- In this array the elements are accessed by using single index.

declⁿ -

$a = [1, 2, 3, 4, 5]$ # declⁿ

print (a[0]) # accessing element at
index zero

② Two Dimensional Array -

- In two dimensional array the elements are stored in matrix form i.e. rows & columns.

- The elements in two dimensional are accessed by two indexes.

* Declⁿ of two dimensional array -

array_name = [rows] [columns] # declⁿ

eg array_name = [[m₁, m₂, ..., m_n], [n₁, n₂, ..., n_n]]

where, m is the row & n is the column of the row/table.

* Accessing elements of two dimensional array - -

$a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]$ # default

— now we can access the first row of array like
 $\text{print}(a[0])$ # where all row accessed by using single index, i.e. row index.

≡ For accessing specific element of row, by using two indexes ---

≡ For accessing 2nd element in first row is

$\text{print}(a[0][2])$

↑ ↑
 row index position index of element into the row.

≡ Accessing all elements of two dimensional array ---

eg $a = [[1, 2, 3], [3, 4, 5], [6, 7, 8]]$

for i in a:
 for j in i:

$\text{print}(i, \text{end}=" ")$
 $\text{print}("\n")$

* Inserting elements into two dimensional array ---

insert() - it is used to insert element in 2D array

Syntax - ~~array~~ ~~name~~ ~~insert~~ (

array-name . insert (index, array elements)

eg a.insert(1, [10, 20, 30])

where, 1 is a index position.

* Update elements into 2D array -

— In This we can add new value to an array as well as change the value at particular index.

eg a = [[1, 2, 3], [4, 5, 6]]
 print(a) # print array before update
 a[0] = [10, 20, 30] # update value at index 0.
 print(a) # print after update

a[1][2] = 100 # update value at position 2nd row 3rd column.
 print(a) # print array after updation.

* deleting element of an array -

— del() - it is used to delete particular index / element ^{of an array} in python.

eg a = [[1, 2, 3], [4, 5, 6]]
 print(a)
 del(a[1]) # delete whole index 1
 print(a)

del(a[0][2]) # delete 3rd element at index 0
 print(a)

String

Page No.

Date

* String - It is a collection of characters or group of characters.

- In python there is no char data type, so a single character also considered as string.

≡ In python for string str is a built-in data type.

≡ In python string is declared following way

Syntax - string-name = "string character"

OR string-name = 'string characters'

OR string-name = """string character"""

eg a = "hello" OR a = 'hello' OR a = """hell

eg a = "hello" # declⁿ of string

print(a) # print string

for i in a:

print(i) # print single-single character of string

print("data-type of a =", type(a)) # display data-type of string

≡ above string can be stored into memory following like way ---

eg a = "hello"

0	1	2	3	4
h	e	l	l	o

← index

≡ in above string we can access single character by using index operator (i.e. [])

like, a[1] - it prints/accesses the character which is at index 1

eg `print(a[1])` # access single character of index position 1

* String Manipulations / String Operations -

① Slicing of String -

- In we can create slices of string by using the indexes (i.e. start index & end index)

- For creating slice we use following syntax -

String-variable [starting index : ending index]

↑
slicing operator

where, starting index - it is starting index of string slice

ending index - it is ending index of string slice (Note -

ending index is always ~~not~~ takes values like, $\text{ending index} - 1$

i.e. ending index = 5 then value becoming $5 - 1 = 4$ i.e. ending index is 4)

colon (:) operator - it is a slicing operator

eg `a = "Sangola"` $a =$

0	1	2	3	4	5	6
S	a	n	g	o	l	a

↑ it is stored like this

- Suppose in we above string we need to create slice like "ango" we need to implement slicing like,

`print(a[1:5])`
← start index ← stop index

— here stop index is 5 because it takes value like $5-1=4$ i.e. 4

≡ For creating slice like "Sang" we implement like ---

`print(a[:4])` # here we not mention starting index, when we do not mention value of starting index, that time it takes index from first element of string.

≡ For creating slice like "gola"

`print(a[3:])` ending index not mentioned when we do not mention ending index that time it takes index upto last element of string

≡ Indexing with Forward step -

ex `a = "Sangola"`

0	1	2	3	4	5	6
S	a	n	g	o	l	a

index

`print(a[1:5:1])`
 ↑ ↑ ↘
 start stop step
 index index value

where, step value indicates step to move next element in string

ex `print(a[1:5:1])` # o/l - ango, here it moves by one to next character

ex `print(a[1:5:2])` # o/l - ag, here it moves by two to next character in string

print(a[::-1]) # o/p - alognas it xerge
 ↑ ↑ ↑
 step given string
 start
 index end index
 not not incremental
 modified

③ calculating length of string -

* len() method - it is used to calculate length / i.e. how much characters are present in given string
syntax `len(string name)`

eg `a = "sanyola"`
`print("The length = ", len(a))`

④ find() method - it is used to find the particular character at which index position in string

syntax `string name.find(character)` eg `a.find('s')`
 eg `a = "sanyola"`
`print("The char find at index = ", a.find('s'))`

⑤ index() method - it is similar to find method

syntax - `string name.index(character)` eg `a.index('s')`

eg `a = "sanyola"`
`print("The char find at index = ", a.index('s'))`

⑥ startswith() & endswith() methods -

* startswith() - it is used to check the given string starts with particular character or not.

syntax - `string name.startswith(character)`
 eg `a = "sanyola"`
`print(a.startswith('s'))`

* endswith() — it is used to check given string ends with particular character or not.

Syntax —

string-name.endswith(character)

eg a = "Sangola"

print(a.endswith(a))

⑦ Repeat given string —

or repeating given string —

Syntax — string-name * repetition no

eg a * 10 # repeats string a 10 times

eg a = "hello"

print(a * 10) OR print("hello" * 10)

⑧ Replace ^{character} word/ in given string —

* replace() method — it is used to replace word/character in given string.

Syntax — string-name.replace("existing word", "replaceable word")

eg a.replace("hello", "hi")

eg. a = "hello students"

print(a.replace("hello", "hi"))

⑨ Testing string membership —

eg a = "Sangola"

~~for~~ if 'n' in a:
 print("true")

else:

print("false")

where, in operator returns true result if character is present in string / member of string

⑩ upper() & lower() methods -

* upper() method - it is used to convert all small letters of string into uppercase letters.

Syntax - string-name.upper()

eg a = "sangola"

print(a.upper()) # convert all letters in uppercase

* lower() method - it is used to convert all letters of string into lowercase

Syntax - string-name.lower()

eg a = "SANGOLA"

print(a.lower()) # convert all letters in lowercase

⑪ split() method -

* split() - it is used to split given string from specified character.

Syntax - string-name.split(specified / splitting character)

eg a = "sangola"

print(a.split("g")) # o/p - san, ola
it splits from character 'g'

⑫ capitalize() method -

it is used to capitalize the first character of string i.e. it convert first character of string in capital form

Syntax - string-name.capitalize()

eg a = "sangola"

print(a.capitalize()) # o/p - Sangola